基于多源异构解析与知识增强的财报“智能问数”助手

摘要

针对上市公司财报场景中非结构化数据难解析、自然语言问答难约束、分析结论难追溯等问题，本文构建了一套面向财报分析的“智能问数”自动化系统。系统以结构化财报数据库为事实底座，以受控 NL2SQL 为查询接口，以研报知识库为分析增强模块，实现了从财报 PDF 解析、数据库构建，到自然语言问答、图表生成、归因解释与结果导出的全流程闭环。

针对任务一，提出基于**多源异构解析**与**勾稽校验**的结构化建模方法。通过 PDF-Word 格式转换、上下文感知的表格抽取、跨页表格合并、单位规范化等步骤, 进一步结合会计恒等式校验、异常识别、联合唯一约束、事务回滚与批次溯源机制, 建立了包含核心业绩指标表、资产负债表、利润表及现金流量表的**结构化财务数据库**。设计了由强结构到弱结构的**四层瀑布流数据映射机制**，在充分利用结构化表格精度的基础上，通过金融专用 **SBERT 语义匹配**弥补非结构化文本检索的不足，实现了核心指标的准确提取与交叉验证。

针对任务二，构建了面向财报问答的分层式智能助手架构，形成“意图识别、槽位补全、NL2SQL 生成、SQL 安全执行、结果序列化、自动图表生成、结论输出”的智能问数链路。通过**意图识别**与**槽位填充**机制准确解析用户提问；引入对话状态跟踪与**上下文管理**维护多轮交互的语义连贯性；基于**关键要素熵值**建立主动类澄清机制提升交互效率；构建图表类型匹配规则库与统计描述模板库，自动生成符合数据特征的可视化结果与定量分析结论。

针对任务三，进一步融合个股与行业研报知识库，建立结构化与非结构化证据协同的**混合检索机制**，增强了系统在复杂财报问题下的可靠性、可解释性与可追溯性。对非结构化研报文本进行**向量索引**与**语义切片**，支持精准的相似段落召回；设计基于大语言模型的**任务规划器**，将复杂复合问题拆解为有序的子任务序列；建立“数据-SQL-研报”的多维追溯链，在输出结果中明确标注引用来源与推理依据，实现从黑盒回答到全过程可解释的跃升。

本文方案避免了通用大模型在无约束条件下直接作答所带来的数值幻觉与口径漂移问题，形成了数据库、问答、批处理、导出与追溯一体化的工程闭环。研究结果可为上市公司财报自动解析、投研辅助决策以及可解释智能问答系统的设计提供参考。

关键字： 财报数据库; 自然语言处理; 知识增强检索; 多意图规划; 多源异构融合

目录

一、 问题重述

1.1 问题背景

随着资本市场信息披露透明度提升与投研分析需求高频化，传统基于人工拆解报表与静态 BI 工具的分析模式已难以满足跨周期、强关联的实时洞察需求，而直接调用大语言模型又面临资源冗余与时效不足的双重困境。为此，构建一套从自然语言输入到可视化分析结论输出的“智能问数”自动化链路，实现上市公司财报数据的零门槛、高效率查询与归因分析，成为提升投研决策精准度的关键突破口。

1.2 问题提出

为构建面向上市公司财报的零门槛智能分析系统，需依次解决以下三个核心问题：

任务一需从非结构化 PDF 财报中自动提取、校验并入库核心财务指标，构建完整规范的结构化数据库。

任务二需搭建具备意图识别、SQL 数据库查询、多轮对话及数据可视化能力的智能助手，实现自然语言到分析结论的全流程自动化响应。

任务三需融合研报知识库，增强助手在多意图规划、输出校验与归因溯源方面的可靠性，确保分析结果可解释、可追溯。

二、 问题分析

本课题旨在构建一套面向上市公司财报的“智能问数”自动化系统，涉及非结构化数据解析、自然语言处理与知识库增强检索三大核心环节。以下针对三个任务的具体难点与建模思路展开深入分析。

2.1 任务一的分析：基于多源异构解析与勾稽校验的结构化建模

任务一要求将非结构化 PDF 财报转化为规范的结构化表格并导入数据库。分析难点在于上交所与深交所财报排版规则各异，表格字段对齐困难。为此，需设计鲁棒的表格定位与字段映射算法，针对不同交易所的命名规则建立正则匹配模版，将数据准确写入资产负债表、利润表、现金流量表及核心业绩指标表。此外，必须建立基于会计勾稽关系的跨表校验模型（如“资产总计 = 负债总计 + 所有者权益总计”），对异常波动数据进行自动识别、合理截断，并标注异常原因，确保最终入库数据的准确性、完整性与逻辑一致性。

2.2 任务二的分析：基于意图识别与上下文关联的交互式查询建模

任务二要求实现从自然语言到可视化结论的自动化响应。其核心在于构建“意图识别—槽位填充—SQL 查询”的映射模型。针对用户模糊提问，需建立基于关键要素熵值的主动澄清引导机制；针对多轮对话场景，引入对话状态跟踪机制以维护上下文的公司实体与时间窗口。在可视化环节，需建立图表类型匹配规则库（如时序数据对折线图、占比数据对饼图），并结合统计描述模板自动生成数据结论，从而实现全流程的无人工干预查询。

2.3 任务三的分析：基于知识增强检索与任务自主规划的归因溯源建模

任务三要求在结构化数据基础上融合个股与行业研报知识库以增强分析深度。首先需对非结构化研报文本进行切片与向量索引构建，实现针对具体问题的相似段落召回。针对多意图复合问题，需设计

基于大语言模型的任务规划器，将复杂问句拆解为有序的子任务序列集。最后，为增强结果可信度，需建立“数据—SQL—研报摘要”的输出追溯链，在结果中明确标注引用来源与推理依据，实现从黑盒回答到可解释归因分析的跃升。

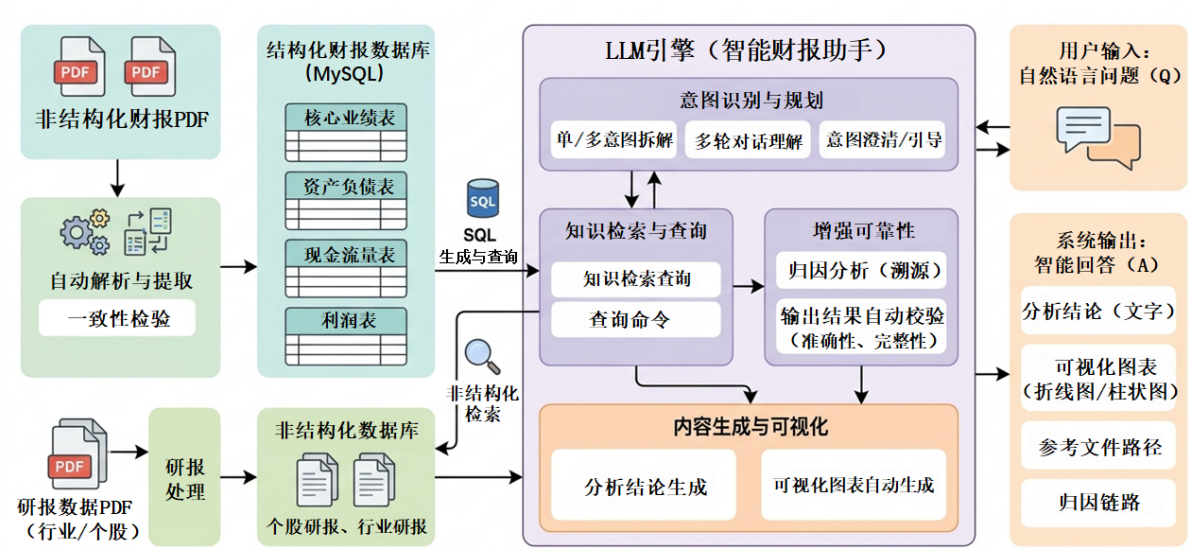

图 1　任务总体流程图

三、 符号说明

变量名	符号说明	单位
q	用户自然语言问题输入	-
$\mathcal{K}_s$	结构化知识库（财务数据库）	-
$\mathcal{K}_u$	非结构化知识库（个股/行业研报）	-
$\mathcal{K}$	异构知识库全集，$\mathcal{K} = \mathcal{K}_s \cup \mathcal{K}_u$	-
$E_s(q)$	问题 q 的结构化证据集合	-
$E_u(q)$	问题 q 的非结构化证据集合	-
$R(e, q)$	候选证据 e 对问题 q 的综合相关度	-
$S_{\text{fusion}}(d)$	候选证据 d 的多路检索融合得分	-
T_i	第 i 个子任务，$T_i = (q_i, p_i, \mathcal{D}_i, \rho_i)$	-
$\mathcal{P}(Q)$	问题 Q 的归因链路集合	-
$\text{Conf}(A)$	最终答案 A 的可信度评分	-

四、 模型假设

为保证模型可实现性与结果可解释性，结合任务一至任务三的建模链路，提出如下关键假设：

1. **数据真实性假设**：公告财报与研报文本来自可信公开渠道，原始数据在采集阶段不被恶意篡改，允许存在少量格式噪声与排版误差。
2. **口径一致性假设**：同一公司同一报告期内，结构化表与文本描述在指标口径上可通过规则映射统一，关键字段（公司、报告期、指标名）可唯一对齐。
3. **时效稳定性假设**：在单轮问答周期内，数据库与检索索引视为静态快照，不考虑查询执行期间外部数据的实时变更。
4. **误差可控假设**：文本抽取、语义匹配与排序融合带来的误差在阈值约束内可控，最终答案可信度可由 $\text{Conf}(A)$ 有效刻画。

五、 模型的建立与求解

5.1 任务一：非结构化财报数据的解析与数据库构建

为确保上市公司财报数据的结构化存储与高效查询，本文设计并实现了一套端到端的自动化数据处理流水线，将非结构化 PDF 格式的财务报告转化为规范化的关系型数据库。该过程涵盖数据预处理、数据映射查找及计算、一致性校验三个核心环节。我们将解析得到的数据处理后写入数据库，最终构建了包含核心业绩指标表、资产负债表、利润表及现金流量表四张核心表的财务数据库，为后续智能助手平台搭建提供了可靠数据基础。

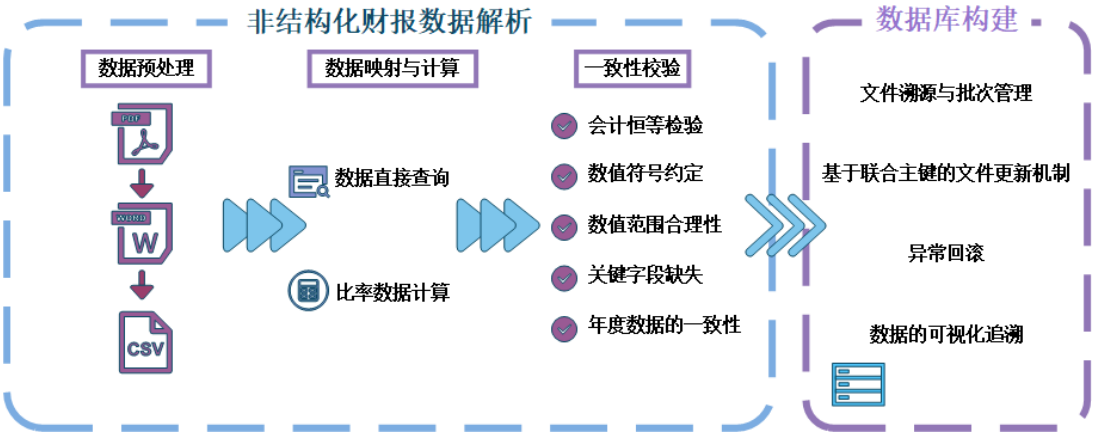

图 2　任务一处理流程架构图

5.1.1 非结构化财报数据的解析

原始财务报告以 PDF 格式呈现，普遍存在版式复杂、表格与正文混排、关键数据跨页拆分等问题，难以直接用于结构化分析与建模。为此，本文先对报告进行解析将其从非结构化的 PDF 格式转换为可编辑的 Word 格式，并从中精准提取财务表格，再经标准化数据清洗与规整处理，完成从非结构化财报到标准化结构化数据的转换。

（1）PDF 格式转为 Word 格式

考虑到 PDF 文件在表格结构识别方面存在较大局限，我们引入 Word 作为中间载体，将 PDF 文档转换为可解析的 Word 格式。相比 PDF，Word 文档能够更清晰地保留表格边界、单元格结构及段落层级关系，从而为后续数据提取提供可靠基础。

财务报告形式上为报告类文字，非扫描件图片，转换可行性非常强。直接从 PDF 解析得到的表格，易出现列数错乱，数字与符号混乱，数据乱序等问题。我们借助 pdf2docx 将 PDF 精准还原为具备原生表格结构的 Word 文档，按版式还原并保留原生表格结构，使表格行列边界、单元格归属、文本对齐关系得到完整保留。

在此基础上再进行表格提取与数据清洗，能够显著提升表格结构还原度，避免因

表 1　PDF 与 WORD 对比

PDF	Word
适合扫描件与图像化表格, 无内在行列结构	原生结构化强, 自带行列标签
文本碎片化, 文字常被拆分为独立文本框	文本连续完整, 便于直接复制使用
复杂排版兼容性差, 易误判表格边界	合并单元格可识别, 准确还原复杂表头
可编辑性差, 提取后需大量人工整理	二次编辑成本极低, 提取后可计算

PDF 非结构化渲染导致的数据错位、符号丢失与数值错乱，确保财务数据的完整性与准确性。

（2）表格抽取与上下文解析

在获得 Word 文档后，对其中的表格与段落进行统一解析。针对财务报表中“表格—标题—单位说明”紧密关联的特点，本文采用**上下文感知的表格抽取策略**，同时获取以下信息：

- 表格内容（数值数据）
- 表格标题（用于识别报表类型）
- 单位信息（用于后续标准化）
- 页码与位置关系（用于跨页识别）

上下文感知的表格抽取策略是指在表格提取时，我们不只是简单将具备行列框架特征的表格进行提取，同时按 Word 文档自上而下的原始阅读顺序，依次提取段落与表格对象，构建连续的文档元素流，严格保留财务报表“标题—单位—表格”的空间邻近关系。在遍历文档流过程中，当识别到“单位：万元”“合并资产负债表”等财务关键段落时，立即缓存为上下文状态，当后续相邻表格出现时，自动将标题与单位绑定至该表格，实现“上下文 → 表格”的精准关联。

（3）跨页表格合并与报表筛选

针对财务报告中常见的跨页表格问题，我们利用分页上下文实现跨页表格的延续性判断与合并，有效解决财务报表因分页导致的表格断裂问题。在依次提取文档中表格及其上下文信息后，通过相邻页码判断、列数一致性校验、表格标题语义匹配、表头行相似性对比四项约束条件，判断后续表格是否为前一表格在分页后的延续。若满足延续条件，则剔除跨页位置重复出现的表头行，将多张分段表格的数据行进行纵向拼接，恢复为完整的单张财务报表结构，避免数据断裂与结构碎片化。

此外，由于原始报告中可能同时包含母公司报表与合并报表，本文基于关键词与语义规则进行筛选，优先保留合并财务报表，以保证数据口径一致性。

（4）单位统一与数据规范化

不同企业财报中常使用“元、千元、万元、亿元”等不同计量单位。为保证数据可比性，定义统一转换函数：

$$x^* = \lambda \cdot x \tag{1}$$

其中，λ 为单位转换系数，x^* 为标准化后的数值（单位统一为“元”）。

同时，对表格数据进行规范化处理，包括：1. 千分位符号去除、2. 括号负数转换（如“(100)”→-100）、3. 百分比数值还原

表 2　部分处理数据展示

stock_code	stock_abbr	net_profit(万元)	•••	repoeriod	r_year
600080	金花股份	3345.950516		FY	2022
600080	金花股份	468.093536		Q1	
600080	金花股份	339.02983		Q3	
600080	金花股份	2396.242714		FY	2023

（5）预处理结果构建

经过上述处理，最终得到完整且数据单位统一的结构化合并报表集合：

$$\mathcal{T}_{clean} = \{\text{合并资产负债表, 合并利润表 合并所有者权益变动表, 合并现金流量表}\} \tag{2}$$

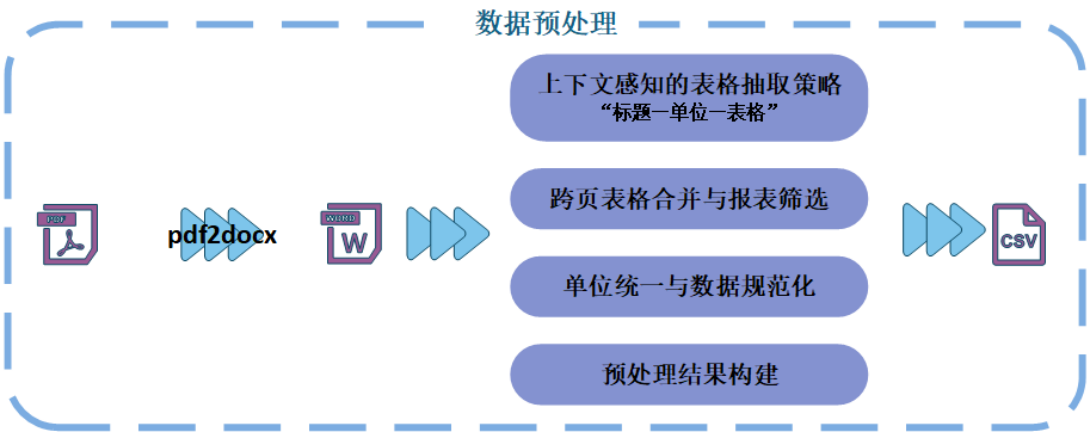

图 3　文件数据预处理流程图

数据映射与计算　针对非结构化财务报告中目标字段分布的结构差异特性，本文构建了结构优先的分级数据映射机制，实现目标字段从结构化表格与非结构化文本中的精准定

位与数值提取。设目标字段集合为 $\mathcal{F} = \{f_1, f_2, \ldots, f_n\}$，原始数据源划分为结构化表格集合 $\mathcal{T}_{tab}$（财报中标准化表格）与非结构化文本集合 $\mathcal{T}_{text}$（Word 格式下财报全文）。

映射函数 $\mathcal{M}$ 遵循由强结构到弱结构的优先级原则，设计四层瀑布流提取策略（表格优先映射层 → 文本上下文检索层 → 字段拓展匹配层 → 指标计算模型），逐层降低检索约束强度、拓展匹配维度，既保证结构化数据的提取精度，又提升非结构化文本中字段的召回率，整体流程如图 1 所示：

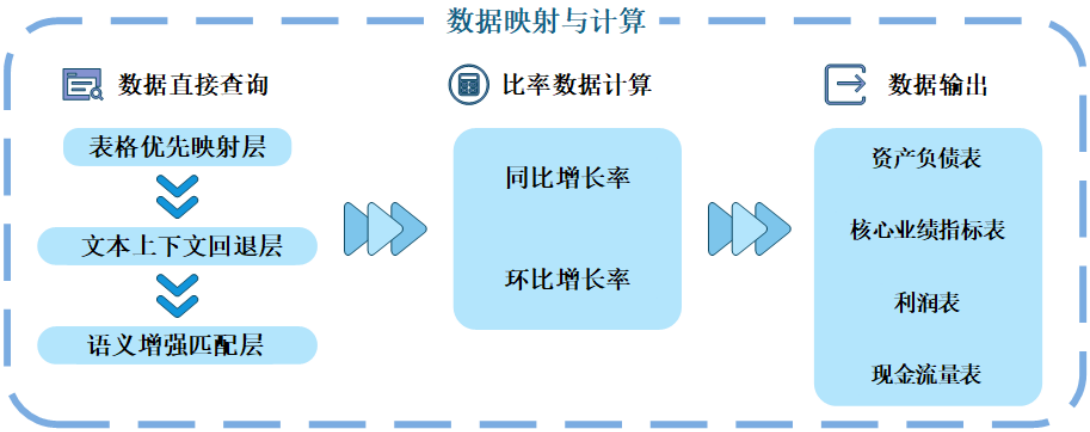

图 4　数据映射与计算流程图

（1）**表格优先映射层**: 该层为强约束检索层，优先从结构化程度最高的财务报表 CSV 文件中完成字段定位，是整个提取机制的核心基础。

- 财务术语标准化处理: 构建金融领域专用的同义词映射表，覆盖关键指标（如每股净资产、营业总收入、净利润等），每类指标关联其标准名称、简称、行业通用别称。对目标字段名称执行标准化清洗，清洗规则包括移除括号内注释、单位后缀、增减幅标识等冗余信息，保留字段核心语义。

- 精确匹配检索: 基于清洗后的标准字段名，在已完成结构化解析的财务表格中执行精确匹配与包含匹配

- 表格结构化解析: 针对财报 CSV 文件的异构格式问题，设计自适应解析策略 (多编码兼容读取, 表头年份自动识别, 报表类型自动分类)

（2）**文本上下文检索层**: 当表格层未匹配到目标字段时，进入文本层检索，从 Word 格式财报全文中基于上下文语义定位数值：

$$f_i = \mathcal{M}_2(\mathcal{T}_{text} \mid \text{context})$$

其中 context 表示关键词上下文语义区域，定义为目标字段关键词所在段落的前 1 段 + 当前段 + 后 1 段构成的文本窗口。

文本检索规则如下：

- 关键词清洗：移除字段名中的空格，生成紧凑关键词；
- 排除性过滤：构建排除关键词列表（母公司、联营企业、少数股东等），若上下文包含该类关键词，则跳过当前匹配结果（避免非合并报表数据干扰）；
- 数值提取：通过正则表达式提取上下文内的数值，支持千分位分隔、负数括号表示、百分比等常见财务数值格式。

在提取时，增加同义词拓展：

- 规则 1：匹配 “即：XXX” 格式，提取括号内的别称；
- 规则 2：匹配 “亦称/又称/别名：XXX” 格式，拆分多别名；

拓展后的同义词集合用于提升文本检索的覆盖率。

（3）**字段拓展匹配层**: 若前两层均未匹配到结果，采用语义匹配完成字段拓展检索，解决字段名称表述差异导致的匹配失效问题。

在语义相似度计算上：采用余弦相似度度量目标字段与候选文本的语义关联度：

$$\text{sim}(q,t) = \frac{q \cdot t}{\|q\|\|t\|}$$

其中 q 为目标字段名称的向量表示，t 为财报候选文本的向量表示，$\text{sim}(q,t) \in [0,1]$，值越大表示语义相似度越高。

在字段拓展的具体方法上,我们使用了**金融专用 SBERT 模型**。并选用 DMetaSoul/sbert-chinese-qmc-finance-v1-distill 模型完成文本向量化，该模型基于金融语料预训练，适配财务术语的语义特征：

- 预计算缓存：对所有表格字段名预计算向量并缓存，避免重复计算；
- 类别约束匹配：结合表格类型分类结果（如资产负债表），仅在同类别候选字段中执行语义匹配，提升匹配精度；
- 阈值筛选：设置相似度阈值 0.85，仅当最大相似度 ≥ 阈值时，返回对应的数值结果。

（4）**指标计算模块**：在计算公式上我们根据会计基础知识，进行如下规定：

同比增长率（Year-on-Year）：

$$g_{\text{yoy}} = \frac{x_t - x_{t-1}}{|x_{t-1}|} \times 100\%$$

其中 x_t 为报告期（t）的字段值，x_{t-1} 为上年同期（$t-1$）的字段值，分母取绝对值避免正负值导致的增长率失真。

环比增长率：

$$g_{\text{qoq}} = \frac{x_t - x_{t-1}^{(p)}}{|x_{t-1}^{(p)}|} \times 100\%$$

其中 $x_{t-1}^{(p)}$ 为上一报告期（如 Q1->Q4（上一年）、Q2->Q1）的字段值。

净利润环比增长率：

$$g_{\text{qoq}} = \frac{x_t - x_{t-1}^{(p)}}{|x_{t-1}^{(p)}|} \times 100\%$$

其中 x_t 为报告期（t）的净利润值，$x_{t-1}^{(p)}$ 为上一报告期（如 Q1->Q4（上一年）、Q2->Q1）的净利润值，分母取绝对值避免正负值导致的增长率失真。

总资产同比增长率：

$$g_{\text{yoy}} = \frac{x_t - x_{t-1}}{|x_{t-1}|} \times 100\%$$

其中 x_t 为报告期（t）的总资产值，x_{t-1} 为上年同期（$t-1$）的总资产值，分母取绝对值避免正负值导致的增长率失真。

净资产收益率（roe）：

$$\text{ROE} = \frac{\text{净利润}}{\text{股东权益合计}} \times 100\%$$

其中分子为报告期归属于母公司的净利润，分母为报告期末股东权益合计，用于衡量股东权益的收益水平。

扣非净利润同比增长率：

$$g_{\text{yoy}} = \frac{x_t - x_{t-1}}{|x_{t-1}|} \times 100\%$$

其中 x_t 为报告期（t）的扣除非经常性损益后净利润值，x_{t-1} 为上年同期（$t-1$）的扣非净利润值，分母取绝对值避免正负值导致的增长率失真。

销售毛利率：

$$\text{销售毛利率} = \frac{\text{营业总收入} - \text{营业成本}}{\text{营业总收入}} \times 100\%$$

其中分子为报告期营业毛利（营业收入扣除营业成本后的剩余部分），分母为报告期营业总收入，反映企业主营业务的盈利空间。

销售净利率：

$$\text{销售净利率} = \frac{\text{净利润}}{\text{营业总收入}} \times 100\%$$

其中分子为报告期净利润，分母为报告期营业总收入，反映企业营业收入转化为净利润的能力。

加权平均扣非净资产收益率：

$$\text{ROE}_{\text{加权}} = \frac{P}{E_0 + \frac{\text{NP}}{2} + E_i \cdot \frac{M_i}{M_0} - E_j \cdot \frac{M_j}{M_0}} \times 100\%$$

其中 P 为报告期扣除非经常性损益后的净利润，E_0 为期初股东权益，NP 为报告期净利润，E_i/E_j 为本期新增/减少的股东权益，M_i/M_j 为对应权益的存续月份，M_0 为报告期总月份，该指标精准反映股东权益的实际收益水平。

而在公式计算上，为构建安全的公式解析引擎，支持 abs 等基础数学函数，我们执行如下流程：

- 公式预处理：移除公式前缀的等号（=），替换字段占位符为实际数值；
- 安全执行：禁用内置函数与危险操作，仅开放白名单函数；
- 校验机制：对同时支持直接提取与公式计算的字段，对比两种方式的结果（误差阈值 10^{-6}），完成数据校验。

一致性校验 为确保数据在进入模型前具有良好的可靠性与逻辑一致性，本文设计了一套多维约束驱动的数据校验机制。

会计恒等式约束为：

$$A \approx L + E \tag{3}$$

容差范围为：

$$|A - (L + E)| \leq \epsilon \tag{4}$$

跨期一致性约束为：

$$\left| \frac{x_t - x_{t-1}}{x_{t-1}} \right| \leq \delta \tag{5}$$

5.1.2 数据库搭建部分

为实现数据来源的全程可追溯与入库数据的唯一性，本文设计了一套基于批次管理机制的数据导入流水线。该流水线不仅负责将解析后的 Excel 结构化数据持久化至 MySQL，更通过严格的事务控制与元数据记录，保障每一次数据变更均可审计、可回滚、可复现。

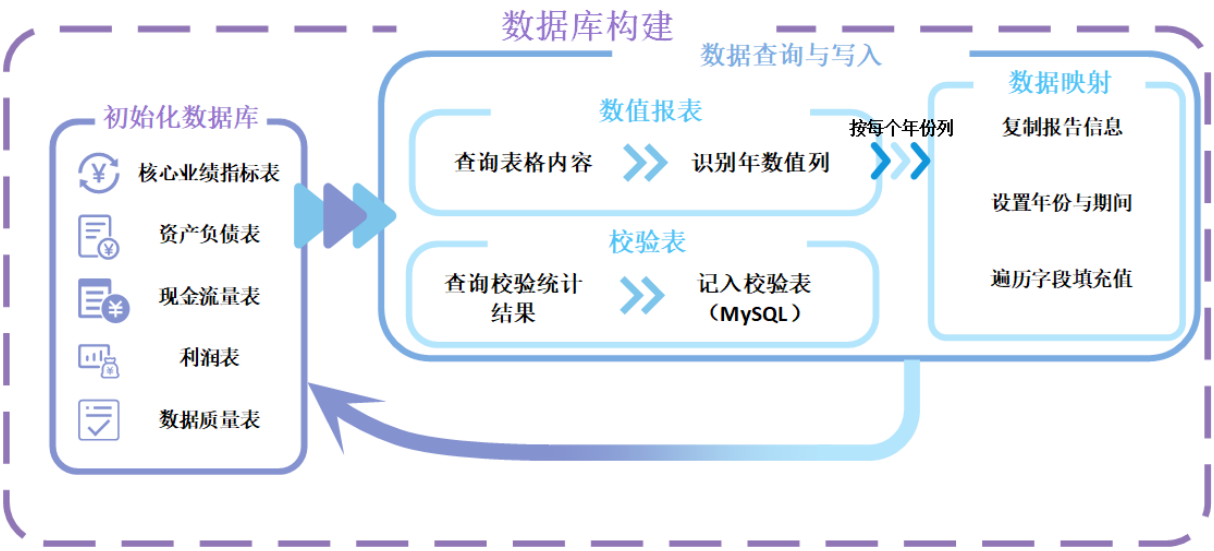

图 5 数据库构建图

文件溯源与批次管理　为实现文件溯源，在文件导入启动时，系统会生成一个全局唯一的批次标识符 `id`，并在相应的审计表中插入一条记录。该记录包含以下关键元数据字段：执行导入的用户标识、源 Excel 文件的完整路径与文件名、文件的 SHA256 哈希值、导入起始时间戳以及初始状态。文件哈希值的引入确保了数据溯源时能够唯一锁定原始文件版本，防止因文件重名或内容篡改导致的数据来源混淆。

基于联合主键的文件更新机制　为杜绝同一家公司同一报告期数据的重复存储，数据库表设计时将（股票代码，报告日期）定义为联合唯一约束。

在数据写入环节，我们没有使用传统的插入或直接替换语句，而是采用了「不存在则插入，存在则更新」的处理逻辑：当待写入的记录，其股票代码与报告期的组合在数据库中已经存在时，系统会自动转为更新操作，用最新解析的数据覆盖原有记录，同时保持标识不变。

这一设计有两个明显的优势：1. 保证了导入数据的唯一性：即使同一份文件被多次导入，也不会产生重复、冗余的数据；2. 保障了数据关联的完整性：避免了传统“先删除再插入”的替换方式，可能引发的数据关联失效问题。

异常回滚　在任意环节发生异常（如字段类型不匹配、网络中断等），事务将整体回滚，同时批次表的状态被更新为 `failed` 并记录完整的异常堆栈信息。通过该设计确保了数据库不会写入残留的脏数据，维持了数据的全局一致性。

数据的可视化追溯　系统为每一条财务数据记录配置了唯一 `id` 字段，实现了数据全生命周期的可追溯管理。通过关联数据导入批次信息，可对任意一条财务记录的来源与变更过程进行完整溯源，具体支持以下追溯场景：如记录来源于哪个 Excel 文件，数据执行导入的操作者是谁等。

5.2 任务二：面向财报的“智能询问”助手构建

在任务一所构建的结构化财报数据库基础之上，本文设计并实现了一款面向上市公司财务数据的“智能问数”助手系统。该系统遵循“自然语言输入 → 意图识别 →SQL 生成 → 数据查询 → 可视化呈现 → 分析结论输出”的全链路自动化架构，具备单轮基础查询、多轮上下文对话、意图澄清引导及动态图表生成四项核心能力，最终以结构化 JSON 格式输出响应内容。

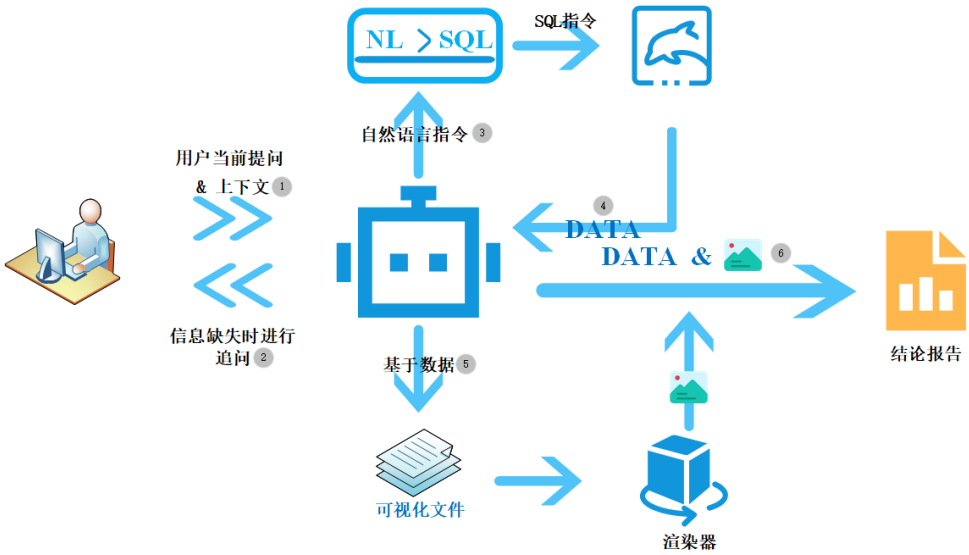

图 6　智能询问助手系统架构图

5.2.1 系统总体架构

针对任务二，为使系统职责清晰、模块化程度高，本文采取了分层设计。将智能询问助手系统拆解成四个协作层：

1. **交互层**：接收用户输入并以流式方式实时回传状态与答案；
2. **理解层**：完成意图识别、上下文继承与澄清决策；
3. **执行层**：NL2SQL 生成、安全校验、数据库查询；
4. **表达层**：自动图表生成、文字结论组织、JSON 结构化输出。

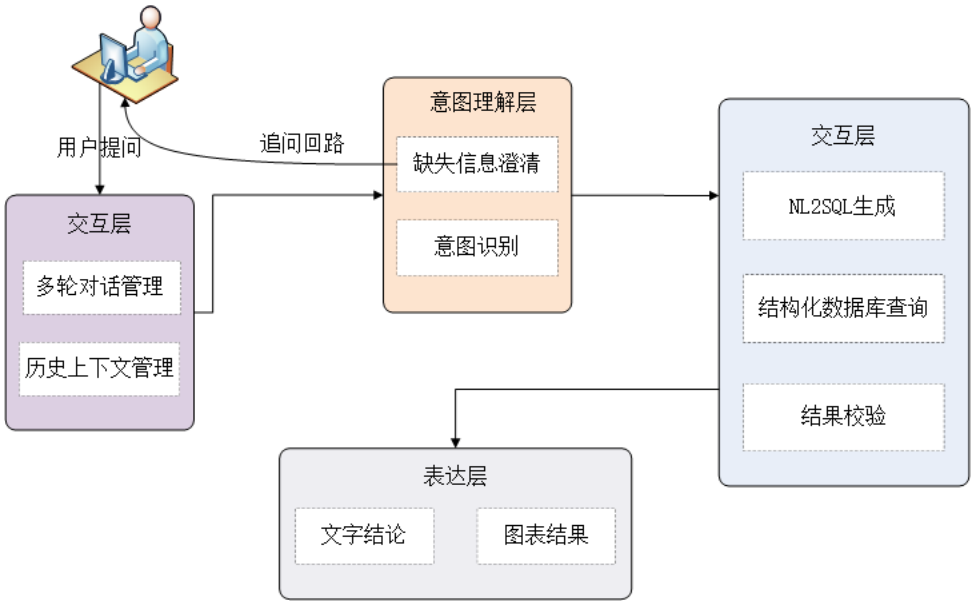

图 7　询问助手协作层结构图

5.2.2 多轮对话与澄清机制

智能助手并不是简单保留聊天记录，而是维护可执行查询所需的**状态变量**。系统维护的核心状态包括公司实体、报告期窗口、指标口径与上一轮查询结果摘要。对于追问类问题，系统优先复用上一轮状态，只更新添加状态变量。通过状态变量的设置让系统能理解上下文语境，处理用户的追问，提高用户的交互效率与体验。

针对用户输入关键信息缺失或意图模糊时，系统触发澄清模块。澄清模块采用“模板保底 +LLM 增强”的混合策略：

1. 首轮优先使用模板追问；
2. 在用户模糊回答或复杂场景下调用 LLM 二次追问；
3. 信息满足执行条件后自动进入 NL2SQL 流程。

通过该策略在评测中有效减少了“错误 SQL”和“答非所问”现象。

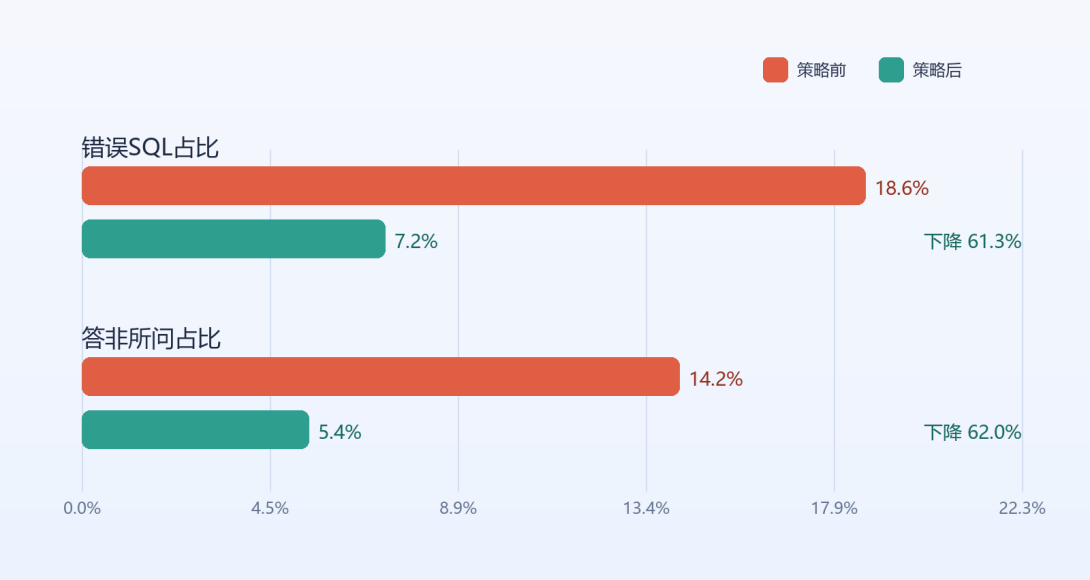

图 8　澄清策略前后效果对比

5.2.3 基于大语言模型的 NL2SQL 生成引擎

本模块旨在将用户自然语言查询自动转化为可执行的结构化查询语句，并在数据库内容基础上生成可解释的分析结论。其核心逻辑为先让大语言模型完成“语义理解与查询规划”，再由数据库返回事实数据，最后由大语言模型负责归纳与解释，从而避免数据伪造提高结果的可核验性。

从形式化角度，给定用户问题 q、数据库模式信息 S 与对话上下文 C，NL2SQL 任务可表示为：

$$\hat{s} = \arg\max_{s \in \mathcal{S}} P(s \mid q, S, C; \theta),$$

其中 $\hat{s}$ 为生成的 SQL，θ 为大语言模型参数。查询执行后得到结果集 $R = \text{Exec}(\hat{s})$，最终分析结论为：

$$\hat{a} = G(q, R, C),$$

其中 $G(\cdot)$ 表示基于结构化结果的分析生成函数。

Algorithm 1: NL2SQL 生成与执行主流程

输入: 用户问题 q, 对话上下文 C, 数据库模式 S, 表白名单 T_{allow}

输出: 回答 A, 元信息 M

```
if ¬IsFinanceQuery(q) then
    return GeneralLLMAnswer(q, C), ∅;
end
P ← BuildPrompt(q, C, S);
J ← LLM_Generate_JSON(P);
if ParseFailed(J) ∨ ¬J.need_sql then
    return GeneralLLMAnswer(q, C), ∅;
end
sql ← J.sql;
if ¬IsSafeSelect(sql, T_allow) then
    return SafeFallbackAnswer(q), {reason: "unsafe"};
end
R ← ExecSQLWithLimit(sql, L_max);
M ← {sql : sql, count : |R|, cols : Cols(R), data : Head(R, k)};
K ← BuildResultContext(M, R);
A ← StreamAnalysisAnswer(q, C, K);
return (A, M);
```

5.2.4 自动图表生成与证据化表达

为满足“可视化输出”要求，系统在 SQL 结果返回后自动进行图表规划。不同于直接让模型执行任意代码，本文采用“**模型决策 + 固定渲染器执行**”方案：

1. 大语言模型根据结果字段与问题语义给出图表规格（如折线/柱状/饼图）；
2. 后端受控渲染器执行绘图并将图片保存到 `result` 目录下；
3. 在流式回复中实时告知图片文件名与保存路径，并结合图表给出结论分析解释。

5.2.5 流式交互与最终输出结构

在用户体验层，系统采用事件驱动流式输出。每轮交互会按阶段返回状态：意图分析、SQL 生成、SQL 执行、图表生成、结论生成。用户可以通过后端的可视化界面，看到系统“正在做什么”。

最终输出与要求格式保持一致：

$$\text{Answer} = \{\text{content},\ \text{image},\ \text{references}\} \tag{6}$$

其中 `image` 为自动生成图表的相对路径列表，`content` 为结合数据证据的分析结论。

5.3 任务三：增强“智能问数”助手的可靠性

5.3.1 总体思路与任务框架

在任务一构建的结构化财报数据库基础上，本文进一步引入个股研报与行业研报，形成面向复杂财报问答场景的可靠性增强模型。任务三的核心目标是：当问题同时涉及数值求解、趋势判断、原因解释与来源追溯时，系统不仅给出答案，还能给出证据基础、形成路径与可信依据。

结合任务一、任务二的处理链路，任务三整体采用“异构知识建模—融合检索—多意图执行—归因回溯—可信校验”的五阶段结构，使回答结果同时满足事实可核验、推理可追踪、来源可定位三项要求。

从问题机理看，任务三的关键难点包括三点：一是结构化财报与非结构化研报在表达形态与证据粒度上显著异构；二是复合问题常包含“筛选—排序—比较—归因”等顺序依赖操作；三是结果必须保留 references 并支持质疑回放。因此，本文将任务三建模为面向异构证据的可解释问答优化问题。

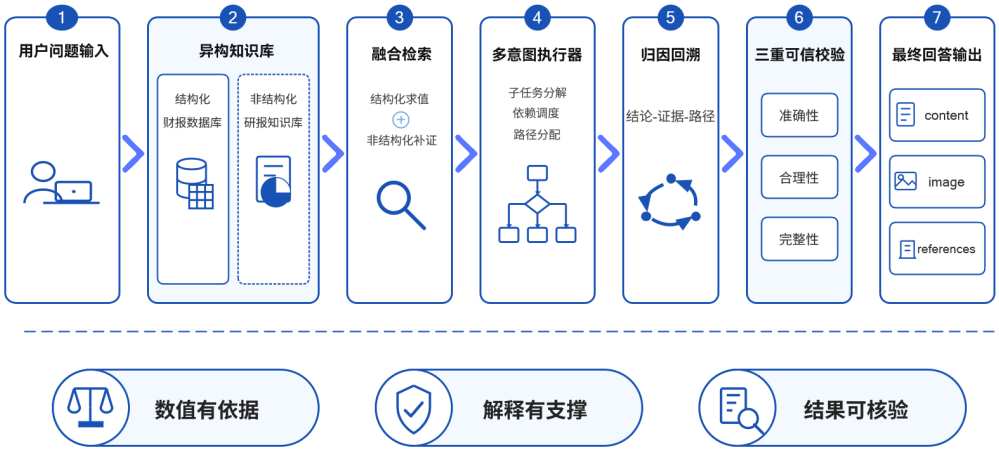

图 9　任务三总框架图

5.3.2 异构知识库的层次化构建

本节先从知识来源构成出发，说明任务三所使用的异构知识库由结构化事实与非结构化证据两部分组成，并明确二者的分工。任务三中的知识库由两部分共同构成：（1）结构化财务数据库：提供利润总额、营业收入、资产负债、现金流等确定性事实；（2）个股研报与行业研报：提供原因解释、行业景气、政策影响及机构判断等补充证据。基于此，异构知识库表示为

$$\mathcal{K} = \mathcal{K}_s \cup \mathcal{K}_u, \tag{5-8}$$

其中，$\mathcal{K}_s$ 表示结构化事实层，$\mathcal{K}_u$ 表示非结构化证据层。前者负责" 事实确定"，后者负责" 语义解释"。

在完成知识库定义后，下一步需要通过问题驱动的检索机制，从海量研报中筛出与当前问题最相关的候选文档。为提高研报知识利用效率，本文采用问题驱动的候选文档筛选机制。设用户问题为 q，候选研报为 d，则其先验相关度定义为

$$S_{\text{doc}}(d,q) \propto \alpha M_{\text{ent}}(d,q) + \beta M_{\text{topic}}(d,q) + \gamma M_{\text{time}}(d,q), \quad \alpha+\beta+\gamma = 1, \tag{5-9}$$

其中，$M_{\text{ent}}(d,q)$、$M_{\text{topic}}(d,q)$、$M_{\text{time}}(d,q)$ 分别对应实体匹配、主题相关与时效性得分。

在文档检索之后，还需要对研报文本做层次化切分，使证据单元既保留局部语义完整性，又便于后续追溯与匹配。在文档表示层，采用层次化语义分块策略。研报文本通常具有“标题—摘要—正文—结论”的层次结构，同一结论往往分布于若干相邻段落中。为保持证据单元内部语义完整性，先依据章节标题完成一级主题分段，再依据段间语义差异完成二级细分。设相邻段落向量为 $\mathbf{h}_i$ 与 $\mathbf{h}_{i+1}$，语义断裂强度定义为

$$PDF\Delta_i = 1 - \cos(\mathbf{h}_i, \mathbf{h}_{i+1}), \tag{5-10}$$

当 $\Delta_i > \tau$ 时判定为新的语义边界，证据单元写为

$$e_j = (x_j, m_j), \tag{5-11}$$

其中 x_j 为文本内容，m_j 为公司、行业、机构、日期、评级等元信息向量。

进一步地，为兼顾语义召回、实体命中与关系约束，本文采用三层索引组织知识库：

- 稠密语义层，负责捕捉跨表述、跨句式的深层语义相似；
- 稀疏关键词层，负责精确命中公司名称、药品名称、政策术语及财务关键词；
- 证据关联层，用于表示“公司—指标—时间—研报—主题”之间的异构联系。

由此，知识库从单纯文本集合提升为由事实层、语义层与关联层共同组成的层次化证据系统。

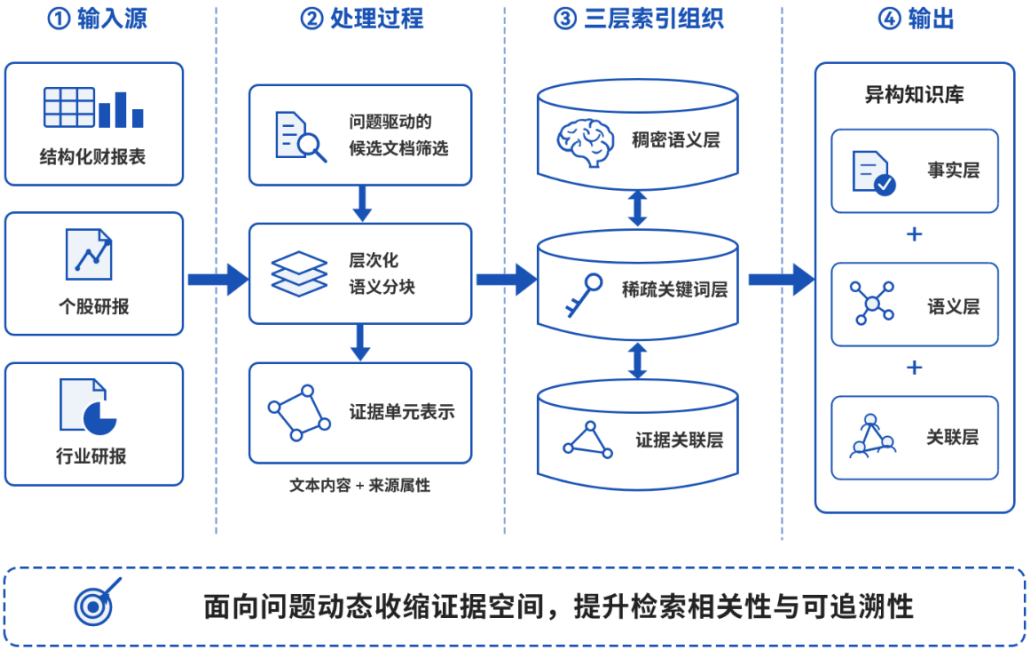

图 10　异构知识库示意图

5.3.3 结构化—非结构化融合检索

财报问答中的证据检索不能仅依赖文本相似度，也不能只依赖数据库精确匹配。本文构建结构化—非结构化双通道融合检索模型，结构化通道负责数值事实求解，非结构化通道负责语义解释召回即从研报知识库中召回与问题语义最相关的背景解释与支撑证据。设结构化证据集合与非结构化证据集合分别为 $E_s(q)$ 与 $E_u(q)$，则综合集合表示为

$$E(q) = E_s(q) \cup E_u(q). \tag{5-12}$$

对于候选证据 e，从语义相关度、实体一致性与来源可信度三个维度定义综合得分：

$$S = R(e,q) = \lambda_1 R_{\text{sem}}(e,q) + \lambda_2 R_{\text{ent}}(e,q) + \lambda_3 R_{\text{src}}(e,q), \quad \lambda_1 + \lambda_2 + \lambda_3 = 1, \tag{5-13}$$

其中 $R_{\text{sem}}(e,q)$ 为语义相关度得分，$R_{\text{ent}}(e,q)$ 为实体一致性得分，$R_{\text{src}}(e,q)$ 为来源可信度得分。该相关度函数将" 相关" 从单纯的语言相似提升为" 语义-实体-来源" 的多维综合评估。并通过多路排序融合统一不同检索器尺度，设候选证据 d 在第 m 路检索中的排名为 $r_m(d)$：

$$S_{\text{fusion}}(d) = \sum_{m=1}^{M} \frac{1}{k + r_m(d)}, \tag{5-14}$$

其中 k 为平滑常数。

为抑制文本证据与财报事实冲突，在融合阶段加入一致性惩罚项。拖候选证据 e 与结构化结果 y_s 在报告期、对象或指标口径上存在冲突，则对其得分施加惩罚：

$$\widetilde{R}(e,q) = R(e,q) - \eta\, \mathbb{I}(e \not\models y_s), \tag{5-15}$$

其中，η 为惩罚系数，$\mathbb{I}(\cdot)$ 为指示函数。

5.3.4 多意图执行与任务编排

任务三中的复合问题通常包含“筛选—统计—比较—归因”的串行目标。本文通过构建多意图执行器模型将复杂问题表示为具有优先级与依赖关系的子任务序列。设原始问题为 Q，则：

$$Q \to \{T_1, T_2, \ldots, T_m\}, \tag{5-16}$$

进一步的将单个子任务表示为

$$T_i = (q_i, p_i, \mathcal{D}_i, \rho_i), \tag{5-17}$$

其中 q_i 为子问题，p_i 为优先级，$\mathcal{D}_i$ 为前置依赖，ρ_i 为执行路径。

根据子任务语义，将执行路径划分为结构化求值路径、非结构化检索路径与混合执行路径：

$$\rho_i = \begin{cases} \text{S}, & \text{结构化求值路径} \\ \text{U}, & \text{非结构化检索路径} \\ \text{H}, & \text{混合执行路径} \end{cases} \tag{5-18}$$

其中 S 表示仅依赖结构化数据库求解，U 表示仅依赖非结构化研报检索，H 表示需要同时调用两者进行融合求解。通过该模型，系统能够根据问题的不同意图自动选择最合适的执行路径。

为体现多意图问题内部的逻辑顺序，本文将任务间关系表示为有向无环图 $G_T = (V_T, E_T)$. 若任务 T_i 依赖于 T_j 的结果，则在图中添加有向边 (T_j, T_i)，表示 T_i 必须在 T_j

完成后才能执行。通过拓扑排序算法确定最终的执行顺序，保证了任务间的正确依赖关系与执行逻辑。

5.3.5 归因链路与可解释回复

为满足归因分析与结果可追溯要求，本文对回答生成过程进行显式建模。设第 i 个子任务的归因单元为

$$P_i = (T_i, E_i, A_i), \tag{5-19}$$

其中 T_i 为子任务，E_i 为支持该子任务的证据集合，A_i 为该子任务的输出答案。则完整归因链路表示为

$$\mathcal{P}(Q) = \{P_1, P_2, \ldots, P_m\}. \tag{5-20}$$

该机制使最终答案由“任务—证据—结论”三元组逐层汇聚生成。

为保证关键结论均有证据支持，定义主张 c_j 的证据支持度：

$$\text{Sup}(c_j) = \max_{e \in \mathcal{E}(c_j)} \left(\omega_1 \text{sim}(c_j, e) + \omega_2 \text{ent}(c_j, e) + \omega_3 \text{src}(e)\right), \tag{5-21}$$

其中 $\mathcal{E}(c_j)$ 为支持主张 c_j 的证据集合，$\text{sim}(c_j, e)$ 为主张与证据的语义相似度，$\text{ent}(c_j, e)$ 为主张与证据的实体一致性得分，$\text{src}(e)$ 为证据来源的可信度得分，$\omega_1, \omega_2, \omega_3$ 分别为三项得分的权重。若 $\text{Sup}(c_j)$ 低于阈值，则该主张不进入最终答案。

当用户发起“依据是什么”类追问时，系统调用解释函，从归因链路中抽取与疑点最相关的局部子图，返回对应的报表字段、研报摘录及推理路径。设质疑问题为

$$Q^\dagger$$

，则解释函数定义为

$$A^\dagger = \Phi(Q^\dagger, \mathcal{P}(Q)), \tag{5-22}$$

其中 $\Phi(\cdot)$ 表示从归因链路中抽取与质疑问题最相关的局部子图，并生成可解释回复的函数。通过该设计，系统能够在面对用户质疑时，提供清晰的证据来源与推理路径，增强回答的透明度与可信度。返回对应的报表字段、研报摘录及推理路径，保证解释与原结论的一致性。

5.3.6 可信校验与结果输出

为进一步保证输出质量，本文在答案生成后执行准确性—合理性—完整性三重可信校验。准确性校验重点核对公司对象、报告期、指标名称、单位及关键数值是否与结构化数据库一致；合理性校验重点判断解释性结论是否具备充分证据支持，是否存在脱离财报事实的强推断；完整性校验重点检查多意图问题中的各个子目标是否均已覆盖。该

校验目标与赛题对任务三的要求完全一致。设三项校验得分分别为 V_a, V_r, V_c，则答案整体可信度定义为

$$\text{Conf}(A) = \mu_1 V_a + \mu_2 V_r + \mu_3 V_c, \quad \mu_1 + \mu_2 + \mu_3 = 1. \tag{5-23}$$

当 $\text{Conf}(A)$ 高于阈值 ξ 时输出最终答案，否则进入保守修正或证据补检阶段。最终答案写为

$$A^* = \text{Verify}\big(\text{Generate}(Q, E(q)), E(q)\big) \tag{5-24}$$

六、 模型评价

6.1 模型的优点

1. 方案覆盖了“PDF 解析—结构化入库—自然语言问答—图表生成—研报增强—结果追溯”的完整链路，流程闭环清晰，便于落地实现。
2. 采用“结构化优先、文本补充”的分层映射机制，并结合会计恒等式与联合约束校验，能够有效降低财报字段错位、漏提和口径不一致问题。
3. 通过意图识别、槽位补全、NL2SQL 与可视化模板联动，提升了自然语言交互的自动化程度，减少了人工干预。
4. 引入研报知识库与证据追溯机制后，模型不仅能给出答案，还能说明答案来源，增强了结果的可解释性与可信度。

6.2 模型的缺点与改进

1. 系统对财报版式和表格质量较为敏感，若原始 PDF 存在严重扫描噪声、跨页断裂或排版异常，解析精度仍会下降。
2. 方案中包含较多规则、阈值和模板设计，虽然可控性较强，但维护成本较高，面对新行业、新披露格式时需要持续调整。
3. 语义检索与大模型规划环节仍可能出现召回偏差或推理漂移，后续可通过更强的领域微调、约束解码和人工反馈闭环进一步优化。

参考文献

[1] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]

[2] Cormack G V, Clarke C L A, Büttcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods[C]

[3] Moreau L, Missier P. PROV-DM: The PROV Data Model[S]. W3C Recommendation, 2013.

[4] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]

[5] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond[J]. Foundations and Trends in Information Retrieval, 2009.

[6] Deng N, Chen Y, Zhang Y. Recent Advances in Text-to-SQL: A Survey of What We Have and What We Expect[C]

[7] Zhong V, Xiong C, Socher R. Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning[EB/OL]

附录 A　附录：主要 Python 代码

1.1　任务一：

Listing 1: 数据公式计算模块部分代码

```
  import os
import pandas as pd
import re
from typing import Dict, Optional, Tuple, List

class RatioCalculator:
    INTERNAL_HELPER_FIELDS = set()
    def __init__(self, raw_data_path: str, config_excel: str, all_data_map: Dict = None):
        """
        :param raw_data_path: 数据提取器输出的中间Excel文件
        :param config_excel: 附件3 Excel 路径
        :param all_data_map: 所有报告数据的映射（用于跨表格计算）
        """
        self.raw_df = pd.read_excel(raw_data_path)
        if '备注' in self.raw_df.columns:
            self.raw_df['备注'] = self.raw_df['备注'].astype(object)

        self.config = pd.read_excel(config_excel, sheet_name=None)
        self.all_data_map = all_data_map if all_data_map else {}

        self.filename = os.path.basename(raw_data_path)
        self.report_year, self.report_period = self._parse_filename(self.filename)
        self.prior_year = self.report_year - 1

        print(f"处理文件: {self.filename}")
        print(f"报告年份: {self.report_year}, 报告期: {self.report_period}, 上年:
            {self.prior_year}")

    def _build_data_map(self) -> Dict[str, float]:
        """构建本期和上期数据的变量映射表"""
        data_map = {}

        for _, row in self.raw_df.iterrows():
            field_id = row['字段名称']
            cn_raw = str(row['中文名称'])
            cn_clean = self._normalize_name(cn_raw)

            val_curr = row.get(f'{self.report_year}年数值')
            val_prior = row.get(f'{self.prior_year}年数值')

            if self._is_ratio_field(cn_raw):
```

```
            continue

        if pd.notna(val_curr):
            v = float(val_curr)
            data_map[field_id] = v
            data_map[cn_raw] = v
            data_map[cn_clean] = v
            data_map[cn_clean.replace('-', '_').replace('（', '_').replace('）', '_')] = v

        if pd.notna(val_prior):
            v = float(val_prior)
            data_map[f'{field_id}_prior'] = v
            data_map[f'{cn_raw}_prior'] = v
            data_map[f'{cn_clean}_prior'] = v
            data_map[f'{cn_clean.replace("-", "_").replace("（", "_").replace("）",
                "_")}_prior'] = v
    return data_map


if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(description="财务比率计算器")
    parser.add_argument("--input_dir", required=True, help="输入目录")
    parser.add_argument("--config", required=True, help="附件3配置文件路径")

    args = parser.parse_args()

    if not os.path.exists(args.config):
        print(f"错误：找不到附件3文件 -> {args.config}")
        exit(1)

    if not os.path.exists(args.input_dir):
        print(f"错误：找不到输入文件夹 -> {args.input_dir}")
        exit(1)

    calculate_all(args.input_dir, args.config)
    print("\n=== 批量处理完成 ===")
```

1.2 任务二

Listing 2: 智能体部分主代码

```
import os
import time
import base64
```

```
import asyncio
import re
import traceback
from typing import Any, List
from openai import AsyncOpenAI
from app.core.config import settings
from app.core.ai_logger import ai_logger
from app.services.clarification_service import ClarificationService
from app.services.chart_service import chart_service
from app.services.task_planner_service import planner_service
from app.services.nl2sql_service import nl2sql_service
from app.services.prompts.prompts_config import PromptsLoader
try:
import dashscope
from dashscope.audio.asr import Recognition, RecognitionCallback, RecognitionResult
except ImportError:
dashscope = None

class AIService:
def __init__(self):
self.kimi_client = AsyncOpenAI(
api_key=settings.MOONSHOT_API_KEY or "placeholder",
base_url="https://api.moonshot.cn/v1",
)

self.deepseek_client = AsyncOpenAI(
api_key=settings.DEEPSEEK_API_KEY or "placeholder",
base_url="https://api.deepseek.com/v1",
)

self.kimi_model = "kimi-k2.5"
self.deepseek_model = "deepseek-reasoner"

# 初始化 ClarificationService
self.clarification_service = ClarificationService()

async def get_image_description(self, file_path: str) -> tuple[str, str | None]:
try:
with open(file_path, "rb") as f:
file_content = f.read()

file_object = await self.kimi_client.files.create(
file=(os.path.basename(file_path), file_content),
purpose="image"
)
file_id = file_object.id
```

```
with open(file_path, "rb") as image_file:
base64_image = base64.b64encode(image_file.read()).decode('utf-8')

response = await self.kimi_client.chat.completions.create(
model=self.kimi_model,
messages=[
{
"role": "user",
"content": [
{"type": "text", "text": "这张图片里有什么？请详细描述。"},
{"type": "image_url", "image_url": {"url": f"{base64_image}"}}
],
}
],
)
return response.choices[0].message.content, file_id
except Exception as e:
return "无法描述图片内容。", None

async def get_video_description(self, file_path: str) -> tuple[str, str | None]:
try:
with open(file_path, "rb") as f:
file_object = await self.kimi_client.files.create(
file=f,
purpose="video"
)
file_id = file_object.id

content_list = [
{"type": "video_url", "video_url": {"url": f"ms://{file_id}"}},
{"type": "text", "text": "请描述这个视频的内容。"}
]

response = await self.kimi_client.chat.completions.create(
model=self.kimi_model,
messages=[{"role": "user", "content": content_list}],
)
return response.choices[0].message.content, file_id
except Exception as e:
return "无法描述视频内容。", None
```

1.3 任务三：

Listing 3: 部分核心代码

```
from __future__ import annotations
```

```
import hashlib
import re
from dataclasses import dataclass
from datetime import datetime
from pathlib import Path
from typing import Any, Iterable, Optional

import chromadb
import pdfplumber
import pymysql
from chromadb.utils import embedding_functions
from openpyxl import load_workbook
from pypdf import PdfReader

from app.core.database import get_db_connection
from app.services.question1_company_reference import (
    load_company_reference,
    normalize_stock_code,
)


BACKEND_DIR = Path(__file__).resolve().parents[2]
FORMAL_DATA_DIR = BACKEND_DIR / "正式数据"
ATTACHMENT5_DIR = FORMAL_DATA_DIR / "附件5: 研报数据"
STOCK_REPORT_DIR = ATTACHMENT5_DIR / "个股研报"
INDUSTRY_REPORT_DIR = ATTACHMENT5_DIR / "行业研报"
STOCK_META_PATH = ATTACHMENT5_DIR / "个股_研报信息.xlsx"
INDUSTRY_META_PATH = ATTACHMENT5_DIR / "行业_研报信息.xlsx"

COLLECTION_NAME = "attachment5_reports"
TOP_LEVEL_TYPE_MAP = {
    "stock": "task3_stock_report",
    "industry": "task3_industry_report",
}
```